

```
aes_586 8124 1164 - Live 0x96a5000
aes_generic 27484 1 aes_586, Live 0xa366000
ecb 2524 1 - Live 0x9981000
biodet_atnc 8356 1 - Live 0xd95d6000
ppdev 6688 0 - Live 0xd9578000
vboxnetadp 78760 0 - Live 0xa0ba000
vboxnetflt 85268 0 - Live 0xd03ba000
vboxdrv 121608 1 vboxnetflt, Live 0xd08ff000
dm_crypt 12828 0 - Live 0xf827e000
bridge 47952 0 - Live 0xd9732000
stp 2272 1 bridge, Live 0xd971c000
bnep 12090 2 - Live 0xd9713000
snd_hda_codec_idt 88644 1 - Live 0xf96c4000
snd_hda_intel 20684 4 - Live 0xf9696000
snd_hda_codec 75708 2 snd_hda_codec_idt, snd_hda_intel, Live 0xf9674000
snd_hweap 7200 1 snd_hda_codec, Live 0xf9653000
```

Another important file is `/etc/modprobe.d/blacklist`. You may also check other files in this folder. This file (which contains a list) allows you to block some of the modules from getting loaded, thus paving the way to use alternative drivers. Sometimes a fake module may also try to block your 'free access' to a device. You can use this file to block the same from getting loaded. Here is a sample of an entry, for your reference:

```
# evbug is a debug tool that should be loaded explicitly
blacklist evbug

# these drivers are very simple, the IBD drivers are usually preferred
blacklist usbmouse
blacklist usbkbd

# replaced by e100
blacklist eepro100

# replaced by tulip
blacklist de4x5

# causes no end of confusion by creating unexpected network interfaces
blacklist eth1394

# snd_intel8x0m can interfere with snd_intel8x0, doesn't seem to support much
# hardware on its own (Ubuntu bug #2011, #6810)
blacklist snd_intel8x0m

<<<OUTPUT TRUNCATED>>>
```

Having discussed these aspects, let's write a module that does more functions than the first one we wrote (refer to Segment 3 for the first tutorial):

Open your editor and write the following lines of code:

```
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/stat.h>
```

```
MODULE_LICENSE("GPL v2");
MODULE_AUTHOR("Aasis Vinayak PG");
```

```
static short int short_integer = 0;
static int integer = 100;
static long int long_integer = 999999;
static char "string_entry" = "GNU/Linux";
static int integerArray[2] = { -1, -1 };
static int arr_arg = 0;
```

```
module_param(short_integer, short, S_IRUSR | S_IWUSR | S_IRGRP |
S_IWGRP);
MODULE_PARAM_DESC(short_integer, "A short integer");
module_param(integer, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IWOTH);
MODULE_PARAM_DESC(integer, "An integer");
module_param(long_integer, long, S_IRUSR);
MODULE_PARAM_DESC(long_integer, "A long integer");
module_param(string_entry, charp, 0000);
MODULE_PARAM_DESC(string_entry, "A String");
module_param_array(integerArray, int, &arr_arg, 0000);
MODULE_PARAM_DESC(integerArray, "An array of integers");
```

```
static int __init tutorial1_init(void)
{
    int i;
    printk(KERN_INFO "Welcome to another day of this Voyage\n");
    printk(KERN_INFO "short_integer is a short integer: %hd\n", short_integer);
    printk(KERN_INFO "integer is an integer: %d\n", integer);
    printk(KERN_INFO "long_integer is a long integer: %ld\n", long_integer);
    printk(KERN_INFO "string_entry is a string: %s\n", string_entry);
    for (i = 0; i < (sizeof integerArray / sizeof (int)); i++)
    {
        printk(KERN_INFO "The integerArray[%d] = %d\n", i, integerArray[i]);
    }
    printk(KERN_INFO "has %d arguments\n", arr_arg);
    return 0;
}

static void __exit tutorial1_exit(void)
{
    printk(KERN_INFO "Exiting the module\n");
}

module_init(tutorial1_init);
module_exit(tutorial1_exit);
```

As you can see in the code, after including the relevant header files (necessary for compiling) and declaring the variables, we start the kernel specific code portion. The next part of the code is used for handling the parameter. And we have already seen why we use `_init` and `_exit`. Even if you are a beginner, I am sure that you would have guessed what this module is doing—we have written a module that can receive values as parameters!

Now we need to write a *make* file. And here is the